Linux Signal-Based Process Controller

Project Documentation

Operating Systems and Systems Programming (25CS2104E) · Section 4 · Team 19

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation

Under the esteemed guidance of **Ms. Soumya Enukonda**

Roll No.	Team Member	Individual Responsibility
2520030419	S. Teja	Design and implementation of the signal-based process controller
2520030418	A. Charanteja	Linux execution, signal testing, and GitHub management
2520030475	E. Ashwith Reddy	Documentation, output verification, and presentation

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives and Scope
5. Existing System and Its Limitations
6. Proposed System
7. Theoretical Background
8. System Requirements
9. System Design
10. Workflow
11. Implementation Approach
12. Source Code
13. Build and Usage Guide
14. Testing
15. Results
16. Impact

17. Limitations
18. Conclusion
19. Future Scope
20. References
21. Appendix

1. Abstract

The Linux Signal-Based Process Controller is a systems programming utility that manages running processes from the command line using POSIX signals. It can pause a process with SIGSTOP, resume it with SIGCONT, terminate it gracefully with SIGTERM, force-kill it with SIGKILL, and restart it by relaunching the original command. In doing so it replaces the ad-hoc use of the `kill` command and shell job control with one consistent tool.

The controller is written in C and is built on the system calls `fork()`, `execvp()`, `kill()`, `waitpid()` and `sigaction()`. Rather than trusting that a signal worked, it verifies every action by reading the real process state from the `/proc` filesystem. This makes the project a compact, working demonstration of how the Linux kernel delivers signals, how a parent process supervises a child, and how process states change over a process's life.

The tool was developed and tested on Linux (Ubuntu), compiled with GCC, and exercised against sample long-running processes such as a counter loop and `sleep 1000`. Testing covered the normal paths (pause, resume, terminate, restart), the escalation path (SIGTERM followed by SIGKILL for stubborn processes) and error cases such as invalid or non-existent PIDs and insufficient permission.

Keywords: Linux, POSIX signals, process control, SIGSTOP, SIGCONT, SIGTERM, SIGKILL, fork, exec, waitpid, /proc.

2. Introduction

Every program running on a Linux system is a process. Operators, developers and students regularly need to stop a process, let it continue, or end it entirely without disturbing the rest of the system. The operating system provides a small but powerful mechanism for this: the **signal**.

A signal is a lightweight, asynchronous notification that the kernel delivers to a process to tell it that some event has occurred. Signals can originate from the keyboard (for example Ctrl+C), from the kernel (for example an illegal memory access), or from another process using the `kill()` system call. Because signals are central to how Unix-like systems manage processes, they are a core topic in Operating Systems and Systems Programming.

This project builds a controller around four of the most important signals:

- **Pause and resume.** SIGSTOP freezes a process and SIGCONT lets it continue from exactly where it stopped.
- **Terminate.** SIGTERM asks a process to exit cleanly, giving it a chance to release resources. SIGKILL ends it immediately and cannot be ignored.
- **Restart.** The controller stops the process and launches a fresh copy using the same command.
- **Verify.** After each action, the controller reads the process state so the user gets confirmation instead of guesswork.

The tools used are the C language on Linux, compiled with GCC and tested on sample long-running processes. The rest of this document explains the problem being solved, the design of the controller, its implementation, the tests performed, and the results obtained.

3. Problem Statement

Managing processes on Linux means remembering signal names, signal numbers and the correct order of commands. Mistakes are easy to make: a mistyped PID can kill the wrong process, and a careless use of SIGKILL can leave files half-written or resources unreleased. A process that has been paused and forgotten, or one that ignores a polite termination request, can leave the user unsure what to do next.

There is no simple, single tool that pauses, resumes, terminates and restarts a process in one place and then confirms what actually happened. Users are forced to combine `kill`, `ps`, `top`, shell job control and manual re-launching.

This project therefore sets out to build a process-control utility that:

1. Sends the proper Linux signal to a target process for each requested action.
2. Checks the state of the process before and after the action.
3. Restarts the process when needed, without the user retyping the original command.

The problem is directly relevant to Operating Systems and Systems Programming because it connects processes, signals, `fork` / `exec` and process states in a real Linux environment.

4. Objectives and Scope

4.1 Objectives

The main objectives of the project are:

- To understand how POSIX signals are generated, delivered and handled on Linux.
- To implement pause and resume of an arbitrary process using SIGSTOP and SIGCONT.
- To implement graceful termination with SIGTERM and a forced fallback to SIGKILL after a timeout.
- To implement restart using `fork()` and `execvp()`.
- To validate the PID and the caller's permission before sending any signal.
- To confirm every action by reading the process state from `/proc/<pid>/stat`.
- To avoid zombie processes and to handle errors safely using `errno` and `sigaction()`.

4.2 Scope

The project covers a command-line controller for Linux and Ubuntu systems. It controls a single target process at a time, either one launched by the controller itself or an existing process identified by PID. It works with user-level processes that the invoking user has permission to signal.

4.3 Out of Scope

The following are intentionally left for future work (see Section 19): automatic crash detection (watchdog mode), control of whole process groups or trees, control by process name, CPU and memory limits, action logging, and graphical or remote interfaces.

5. Existing System and Its Limitations

Today, process control on Linux is done with a handful of general-purpose tools used by hand.

Manual use of `kill`. Processes are controlled by typing commands such as `kill -STOP`, `kill -CONT` or `kill -9` followed by a PID. The user must remember the signal names and numbers and type the PID correctly. A single wrong digit sends the signal to a different process.

Shell job control. The shortcuts Ctrl+Z, `fg` and `bg` pause and resume jobs, but they work only for jobs started in the same terminal. They cannot be used to control an arbitrary process that is already running elsewhere.

Overuse of `kill -9`. Because it always works, SIGKILL is often used first. The program receives no warning, so it cannot save data, close files or release locks.

No restart facility. There is no built-in way to restart a process. The user must remember the original command line, with all its arguments, and relaunch it manually.

No feedback. After sending a signal the user gets no confirmation. A separate `ps` or `top` command is needed to check that the signal had the intended effect.

Aspect	Existing approach	Limitation
Pause / resume	`kill -STOP/-CONT`, Ctrl+Z, `fg`, `bg`	Job control limited to one terminal; signal names must be remembered
Termination	`kill -9` often used first	No chance for clean shutdown; data loss possible
Restart	Manual relaunch	Original command must be remembered and retyped
Verification	Separate `ps` / `top`	No automatic confirmation of the result
Error handling	Raw shell error messages	Easy to target the wrong PID

6. Proposed System

The proposed solution is a C utility for Linux and Ubuntu that controls a process through POSIX signals from a single command-line interface. The controller accepts a command and a target, translates the command into the correct signal, delivers it with `kill()`, and then reports the resulting process state.

6.1 Key Features

- **Pause and resume** any target process using SIGSTOP and SIGCONT.
- **Graceful termination** with SIGTERM, falling back to SIGKILL after a timeout so that unresponsive processes are still removed.
- **Restart** by stopping the process and relaunching it with `fork()` and `execvp()`.
- **Validation** of the PID and of permission before any signal is sent.
- **Verification** of each action by reading the process state from `/proc`.
- **Clean exit** through `sigaction()` handlers for SIGINT and SIGCHLD, so the controller itself never leaves zombie children behind.

6.2 Advantages over the Existing System

Concern	Existing system	Proposed system
Interface	Several commands to remember	One tool with simple verbs
Safety	Direct SIGKILL common	SIGTERM first, SIGKILL only when needed

Concern	Existing system	Proposed system
Restart	Manual	Built in
Feedback	None	State confirmed from `/proc`
Errors	Wrong PID goes unnoticed	PID and permission validated first

7. Theoretical Background

This section summarises the operating-system concepts on which the controller is built.

7.1 Processes

A process is a running instance of a program. The kernel identifies each process by a unique **process ID (PID)** and keeps track of its parent through the **parent process ID (PPID)**. A process is created when an existing process calls `fork()`, which duplicates the caller, and it usually then calls one of the `exec` family of functions to replace its memory image with a new program.

7.2 Process States

At any moment a Linux process is in one of several states. The state letter appears in `ps` output and in the third field of `/proc/<pid>/stat`.

State letter	Meaning	Relevance to this project
R	Running or runnable	Expected after resume for a busy process
S	Sleeping (interruptible wait)	Expected after resume for an idle process such as `sleep`
D	Uninterruptible sleep	Cannot be signalled until the wait ends
T	Stopped by a signal	Expected after SIGSTOP
Z	Zombie (exited, not yet reaped)	Must be avoided by calling `waitpid()`

7.3 Signals

A signal is delivered to a process by the kernel. When it arrives, the process may handle it with a custom function, ignore it, or let the default action occur. Two signals are special: **SIGKILL and SIGSTOP can be neither caught nor ignored**, which is why they are reliable

tools for control.

Signal	Number (x86 Linux)	Default action	Can be caught?	Used by the controller for
SIGSTOP	19	Stop the process	No	Pause
SIGCONT	18	Continue a stopped process	Yes	Resume
SIGTERM	15	Terminate	Yes	Graceful stop
SIGKILL	9	Terminate immediately	No	Forced stop
SIGINT	2	Terminate (Ctrl+C)	Yes	Handled by the controller for clean exit
SIGCHLD	17	Ignore	Yes	Tells the parent a child changed state

7.4 The System Calls Used

`fork()` creates a child process that is a copy of the caller. It returns 0 in the child and the child's PID in the parent.

`execvp(file, argv)` replaces the current process image with a new program, searching the `PATH` for it. It returns only if it fails.

`kill(pid, sig)` sends the signal `sig` to the process `pid`. Despite its name it can send any signal. When `sig` is 0 no signal is sent, but the call still performs the existence and permission checks, which makes `kill(pid, 0)` a convenient validator.

`waitpid(pid, &status, options)` lets a parent wait for a child to change state and **reap** it. With the `WNOHANG` option it returns immediately instead of blocking, which allows polling. Reaping prevents zombie processes.

`sigaction(sig, &act, &old)` installs a signal handler. It is preferred over the older `signal()` because its behaviour is precisely defined and portable.

7.5 The /proc Filesystem

`/proc` is a virtual filesystem through which the kernel exposes information about running processes. The file `/proc/<pid>/stat` holds a single line of fields, of which the third is the process state letter. The file `/proc/<pid>/status` presents the same information in a

human-readable form with a line beginning `State:`. If `/proc/<pid>` does not exist, the process does not exist.

7.6 Signal Delivery and Permissions

An unprivileged user may send a signal only to processes owned by the same user. Attempting otherwise makes `kill()` fail with the error `EPERM`. If the PID does not exist, it fails with `ESRCH`. The controller uses these two error codes to give clear, specific messages.

8. System Requirements

8.1 Hardware Requirements

Component	Minimum requirement
Processor	Any x86-64 or ARM64 processor capable of running Linux
Memory	1 GB RAM
Storage	100 MB free disk space

8.2 Software Requirements

Component	Requirement
Operating system	Linux / Ubuntu
Programming language	C
Compiler	GCC
Process information	`/proc` filesystem, `ps` and `top` for verification
Development tool	VS Code
Version control	Git and GitHub

8.3 Functional Requirements

- FR1: The tool shall pause a process when given the `pause` command and a valid PID.
- FR2: The tool shall resume a stopped process when given the `resume` command.

- FR3: The tool shall terminate a process gracefully with SIGTERM and escalate to SIGKILL after a timeout.
- FR4: The tool shall force-kill a process immediately when given the `kill` command.
- FR5: The tool shall restart a process by relaunching its saved command.
- FR6: The tool shall report the process state after every action.

8.4 Non-Functional Requirements

- **Reliability:** no zombie processes shall remain after any operation.
- **Safety:** invalid input shall be rejected without sending any signal.
- **Usability:** error messages shall be clear and name the cause.
- **Portability:** the code shall use only POSIX interfaces plus `/proc`.